SOFTWARE ENGINEERING · CHAPTER 05

Security in Java Web Applications (Spring Security, JWT)

Web application security protects data, users, and services from unauthorized access and common attacks. Spring Security provides authentication, authorization, and security integration, while JWT supports stateless token-based authentication.

Spring Security 7.1.0 | Spring Boot 4.1.1 | JWT · RFC 7519 | JDK 25 LTS

AGENDA

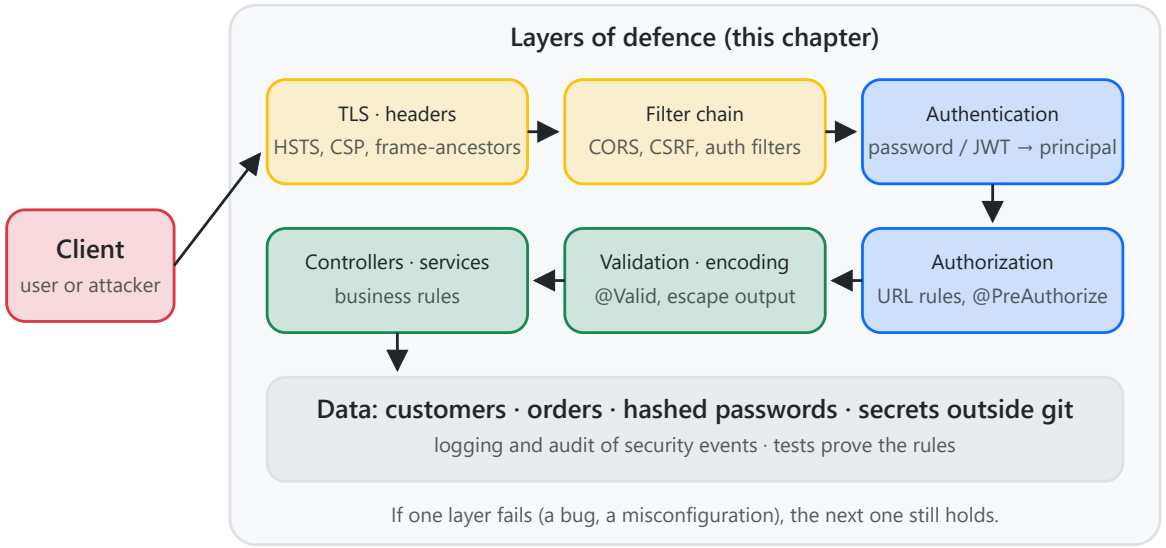
What we will cover

1. Authentication, authorization, and security contexts
2. Password hashing and credential management
3. Roles, authorities, and method-level security
4. HTTP security filters and request protection
5. Session-based and stateless authentication
6. JWT structure, signing, validation, and expiration
7. CSRF, CORS, clickjacking, and security headers
8. Input validation and output encoding
9. Secure secrets and configuration management
10. Logging, testing, and common OWASP risks
11. Chapter Quiz
12. Lab-05

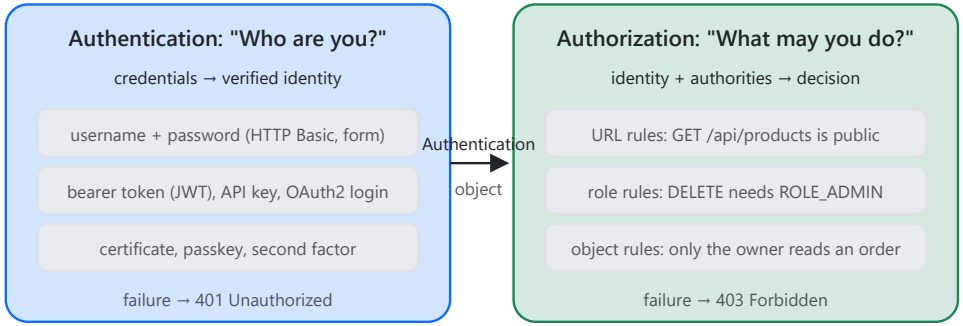
Click any item to jump directly to that topic.

Why every web application needs a security layer

- A web application is reachable by **anyone on the network**: customers, bots, competitors, attackers. Every request is untrusted until proven otherwise.
- Security has three jobs: **who are you?** (authentication), **what may you do?** (authorization), and **how do we stop abuse?** (protection against CSRF, XSS, injection, leaked secrets).
- **Spring Security 7.1** is a chain of servlet filters plus an authorization model that integrates with Spring MVC, method calls and tests.
- **JWT** (JSON Web Token) is a signed, self-contained token that lets a REST API stay *stateless*: no server-side session.
- Principle of the chapter: **defence in depth**. No single control is enough; layer them.



Authentication and authorization are two different questions



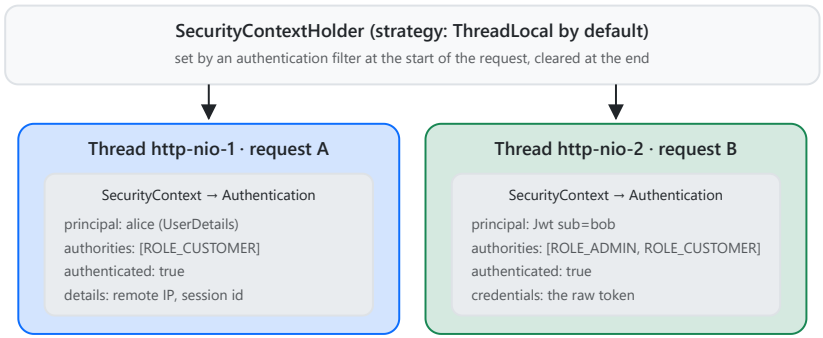
Spring Security: AuthenticationManager does the first, AuthorizationManager the second.

Both read and write the same thing: the SecurityContext of the current request.

Term	Spring Security type	Example
Principal	`UserDetails` / `Jwt`	customer `alice`
Credentials	password, token	`Authorization: Bearer …`
Authority	`GrantedAuthority`	`ROLE_ADMIN`, `orders:read`
Authentication	`Authentication`	principal + authorities + `isAuthenticated()`
Context	`SecurityContext`	holds the `Authentication` for one request

```java
// The two decisions in one controller method
@GetMapping("/api/orders/{id}")
public OrderDto one(@PathVariable long id,
                    Authentication auth) {
    // 1. who? (already done by a filter, or auth == null)
    String user = auth.getName();
    boolean admin = auth.getAuthorities().stream()
        .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
    // 2. what may they do? (better: @PreAuthorize, slide 9)
    Order order = orders.get(id);
    if (!admin && !order.customer().equals(user))
        throw new AccessDeniedException("not your order");
    return OrderDto.from(order);
}
```

The `SecurityContext`: one `Authentication` per request thread



Two requests never see each other's context. Child threads need explicit propagation.

```java
// Three ways to read the current user (prefer the first two)
@RestController
@RequestMapping("/api/me")
public class MeController {

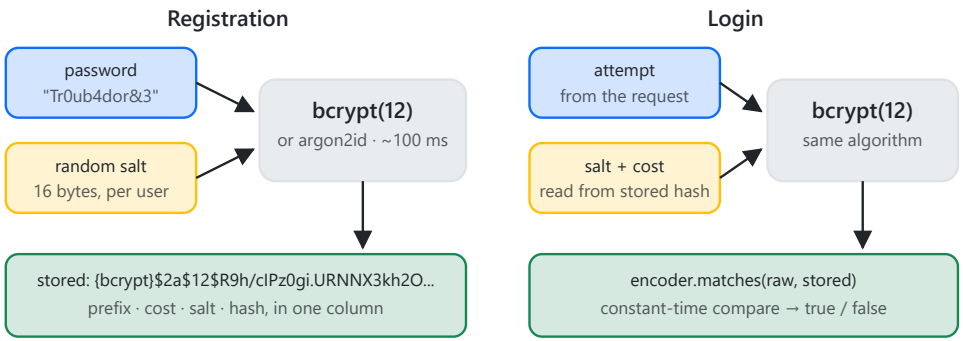
    // 1. injected as a method argument
    @GetMapping
    public Map<String, Object> me(Authentication auth) {
        return Map.of("name", auth.getName(),
                      "roles", auth.getAuthorities());
    }

    // 2. the principal object directly (UserDetails, Jwt, ...)
    @GetMapping("/details")
    public UserDetails details(
            @AuthenticationPrincipal UserDetails user) {
        return user;
    }
}

// 3. anywhere in a service: the static holder (ThreadLocal)
@Service
public class AuditService {
    public void record(String action) {
        Authentication auth = SecurityContextHolder
            .getContext().getAuthentication();
        String who = auth == null ? "anonymous" : auth.getName();
        log.info("audit user={} action={}", who, action);
    }
}
```

⚠ **Threads:** the context is a `ThreadLocal`. Work handed to an executor or `@Async` method starts with an empty context unless you use `DelegatingSecurityContextExecutor` or set `SecurityContextHolder.setStrategyName(MODE_INHERITABLETHREADLOCAL)` (Chapter 01 revisited).

Never store passwords: store slow, salted hashes



MD5 / SHA-256 alone are fast (billions per second on a GPU): useless for passwords.

Salt defeats rainbow tables; the cost factor makes brute force slow; the hash is one-way.

A DB leak then exposes hashes, not passwords.

```java
@Configuration
public class PasswordConfig {

    // Recommended: delegating encoder, bcrypt as default.
    // Hashes carry an {id} prefix, so you can migrate later.
    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories
            .createDelegatingPasswordEncoder();
    }

    // Alternatives when you need to tune the cost:
    PasswordEncoder bcryptStrength12() {
        // strength 10..31; 12 ≈ 250 ms on a laptop
        return new BCryptPasswordEncoder(12);
    }
    PasswordEncoder argon2() {
        // saltLength, hashLength, parallelism, memory KB, iterations
        return new Argon2PasswordEncoder(16, 32, 1, 65_536, 3);
    }
}

// Usage at registration and login
String stored = encoder.encode("Tr0ub4dor&3");
// {bcrypt}$2a$10$...    (60 chars after the prefix)
boolean ok = encoder.matches(attempt, stored);
if (encoder.upgradeEncoding(stored)) {
    // re-hash with the current default and save
}
```

Encoder	Notes
`BCryptPasswordEncoder`	Default, well understood, 72-byte input limit, cost 10 to 12
`Argon2PasswordEncoder`	Memory-hard (GPU resistant), winner of the PHC, needs Bouncy Castle
`Pbkdf2PasswordEncoder`	FIPS-compliant option, many iterations
`{noop}`, MD5, SHA	Demos and legacy only, never in production

Credential management: `UserDetailsService` backed by JPA

```
@Entity @Table(name = "users")
public class UserAccount {
    @Id @GeneratedValue private Long id;
    @Column(unique = true, nullable = false) private String username;
    @Column(nullable = false) private String passwordHash;
    private boolean enabled = true;
    private int failedAttempts;
    private Instant lockedUntil;
    @ElementCollection(fetch = FetchType.EAGER)
    @Enumerated(EnumType.STRING)
    private Set<Role> roles = new HashSet<>();
    // getters, setters, no password field in toString()
}

public enum Role { CUSTOMER, ADMIN }

@Service
public class JpaUserDetailsService implements UserDetailsService {
    private final UserAccountRepository users;
    public JpaUserDetailsService(UserAccountRepository users) {
        this.users = users;
    }

    @Override
    public UserDetails loadUserByUsername(String username) {
        UserAccount u = users.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));
        return User.withUsername(u.getUsername())
            .password(u.getPasswordHash())      // already hashed
            .roles(u.getRoles().stream()        // adds ROLE_ prefix
                    .map(Enum::name).toArray(String[]::new))
            .disabled(!u.isEnabled())
            .accountLocked(u.getLockedUntil() != null
                && u.getLockedUntil().isAfter(Instant.now()))
            .build();
    }
}
```

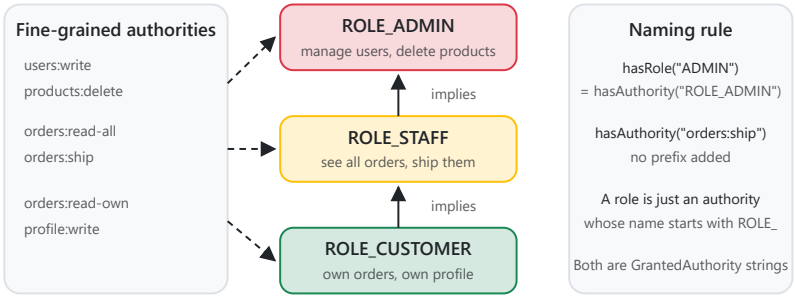
- Spring Security only needs **one bean** from you: a `UserDetailsService`. The `DaoAuthenticationProvider` calls it, then runs `PasswordEncoder.matches()`.
- The generated `UserDetails` carries the **hash**, never the raw password; Spring erases credentials from the `Authentication` after a successful login.
- **Registration** is your code: validate the DTO, apply a password policy, `encoder.encode()`, save, return `201` without echoing the password.
- Use a **generic error** for both unknown user and wrong password (`BadCredentialsException`), otherwise you leak which usernames exist.
- **Lockout** after N failures and **rate limiting** stop online brute force; slow hashing protects the offline case.

```
public record RegisterRequest(
    @NotBlank @Size(min = 3, max = 40)
    @Pattern(regexp = "[a-z0-9_.]+") String username,
    @NotBlank @Size(min = 12, max = 72)
    @Pattern(regexp = "(?=.*\\d)(?=.*[a-z])(?=.*[A-Z]).*",
             message = "needs upper, lower and digit")
    String password) {}

@PostMapping("/api/register")
@ResponseStatus(HttpStatus.CREATED)
public void register(@RequestBody @Valid RegisterRequest r) {
    accounts.create(r.username(),
        encoder.encode(r.password()), Set.of(Role.CUSTOMER));
}
```

Boot's default `spring.security.user.name/password` and the generated password in the log are for the first five minutes of a project only. Replace them with your `UserDetailsService` and they disappear.

Roles, authorities and the role hierarchy



A RoleHierarchy bean saves you from listing every lower role in every rule.

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET, "/api/products/**")
                .permitAll()
            .requestMatchers("/api/register", "/api/auth/login")
                .permitAll()
            .requestMatchers(HttpMethod.DELETE, "/api/products/**")
                .hasRole("ADMIN")                    // ROLE_ADMIN
            .requestMatchers("/api/orders/*/ship")
                .hasAuthority("orders:ship")         // exact name
            .requestMatchers("/api/admin/**")
                .hasAnyRole("ADMIN", "STAFF")
            .requestMatchers("/api/**").authenticated()
            .anyRequest().denyAll());                // safe default
        return http.build();
    }

    // ADMIN gets everything STAFF has, STAFF everything CUSTOMER has
    @Bean
    static RoleHierarchy roleHierarchy() {
        return RoleHierarchyImpl.withDefaultRolePrefix()
            .role("ADMIN").implies("STAFF")
            .role("STAFF").implies("CUSTOMER")
            .build();
    }
}
```

⚠ Order matters: matchers are evaluated top to bottom and the **first match wins**. Put specific rules first and end with `anyRequest()`. Since Spring Security 6 the URL rules apply to every dispatch type (including error and async) unless you say otherwise.

Method security: `@PreAuthorize` and `@PostAuthorize` on services

```java
@Configuration
@EnableMethodSecurity      // enables @PreAuthorize & co. via AOP proxies
public class MethodSecurityConfig {}

@Service
public class OrderService {
    private final OrderRepository orders;
    public OrderService(OrderRepository orders) { this.orders = orders; }

    // Admins see everything; a customer only their own orders.
    // #customerId is the method parameter, principal is the logged-in user
    @PreAuthorize("hasRole('ADMIN') or #customerId == authentication.name")
    public List<Order> history(String customerId) {
        return orders.findByCustomer(customerId);
    }

    // Owner check on the *returned* object
    @PostAuthorize("hasRole('ADMIN') "
                 + "or returnObject.customer == authentication.name")
    public Order get(long id) {
        return orders.findById(id).orElseThrow(OrderNotFound::new);
    }

    @PreAuthorize("hasAuthority('orders:ship')")
    public void ship(long id) { orders.markShipped(id); }

    // Filter a collection instead of rejecting the whole call
    @PostFilter("hasRole('ADMIN') "
              + "or filterObject.customer == principal.username")
    public List<Order> recent() { return orders.findTop50(); }

    // Call a bean for rules that don't fit one line of SpEL
    @PreAuthorize("@orderGuard.canCancel(#id, authentication)")
    public void cancel(long id) { orders.cancel(id); }
}
```

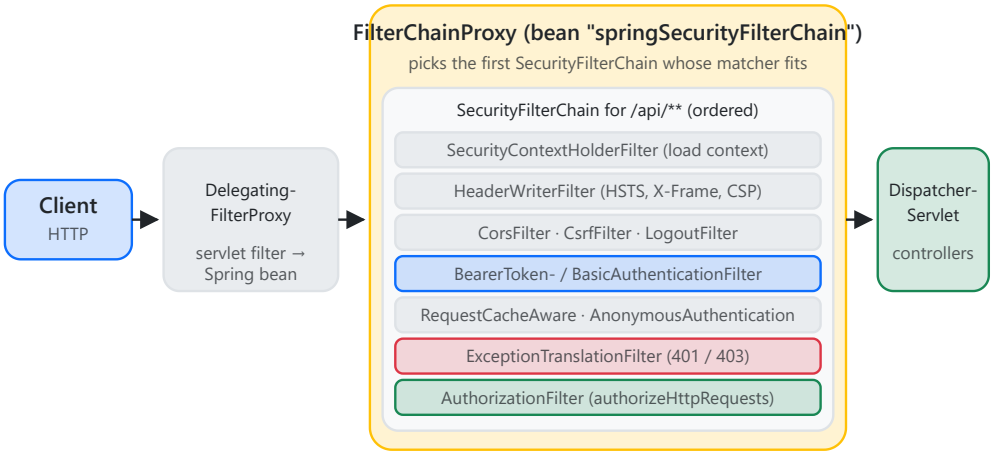
Annotation	When	Use for
`@PreAuthorize`	before the call	roles, argument checks
`@PostAuthorize`	after, sees `returnObject`	owner of the loaded entity
`@PreFilter` / `@PostFilter`	filter collections	hide others' rows
`@Secured` / `@RolesAllowed`	before, roles only	legacy / Jakarta annotation

SpEL name	Meaning
`authentication`	the current `Authentication`
`principal`	`authentication.getPrincipal()` (UserDetails or Jwt)
`#param`	method argument by name
`returnObject`, `filterObject`	result / element being filtered
`@bean.method(...)`	delegate to a Spring bean

⚠ Same proxy rules as `@Transactional` (Chapter 02): annotate **public** methods of a **bean**; self-invocation bypasses the check.

💡 URL rules are the coarse outer fence; method rules protect the business operation no matter which controller, scheduler or message listener calls it. Use both.

Spring Security is a chain of servlet filters



Each filter may stop the chain (reject) or call the next one. Responses flow back through the same filters.

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    // One chain for the JSON API ...
    @Bean @Order(1)
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http.securityMatcher("/api/**")
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.GET, "/api/products/**")
                    .permitAll()
                .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults())
            .csrf(csrf -> csrf.disable())      // stateless API only
            .sessionManagement(s -> s
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS));
        return http.build();
    }

    // ... and another one for the server-rendered admin UI
    @Bean @Order(2)
    SecurityFilterChain web(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/css/**", "/login").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .formLogin(form -> form.loginPage("/login"))
            .logout(Customizer.withDefaults());
        return http.build();
    }
}
```

💡 The lambda DSL (`http.x(x -> ...)`) is the only style in Spring Security 7; `WebSecurityConfigurerAdapter` and the `and()` chaining are gone. Add `logging.level.org.springframework.security=TRACE` to see which filter rejected a request.

Request protection: entry points, access denied and your own filter

```
// Consistent problem+json errors for both security failures
@Bean
SecurityFilterChain api(HttpSecurity http,
                        ObjectMapper json) throws Exception {
    http.securityMatcher("/api/**")
        .authorizeHttpRequests(a -> a.anyRequest().authenticated())
        .exceptionHandling(ex -> ex
            // not authenticated (or bad token) → 401
            .authenticationEntryPoint((req, res, e) ->
                problem(res, json, HttpStatus.UNAUTHORIZED, e))
            // authenticated but not allowed → 403
            .accessDeniedHandler((req, res, e) ->
                problem(res, json, HttpStatus.FORBIDDEN, e)))
        // a custom filter placed before authorization
        .addFilterBefore(new RequestIdFilter(),
                         AuthorizationFilter.class);
    return http.build();
}

static void problem(HttpServletResponse res, ObjectMapper json,
                    HttpStatus status, Exception e) throws IOException {
    ProblemDetail pd = ProblemDetail.forStatusAndDetail(status,
        status == HttpStatus.UNAUTHORIZED
            ? "Authentication required" : "Access denied");
    res.setStatus(status.value());
    res.setContentType(MediaType.APPLICATION_PROBLEM_JSON_VALUE);
    json.writeValue(res.getOutputStream(), pd);
}

// A filter that runs once per request (not again on forwards)
public class RequestIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String id = UUID.randomUUID().toString();
        MDC.put("requestId", id);
        res.setHeader("X-Request-Id", id);
        try { chain.doFilter(req, res); } finally { MDC.remove("requestId"); }
    }
}
```

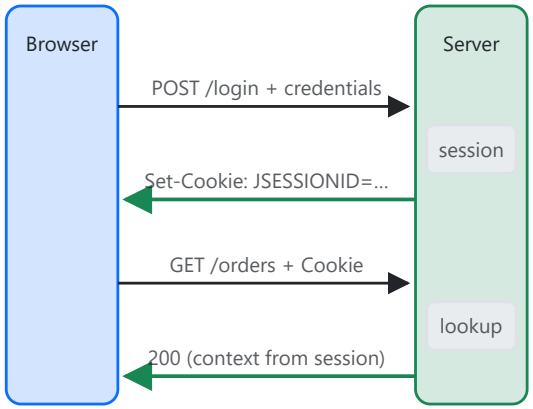
Situation	Status	Handled by
No credentials, rule needs a user	`401`	`AuthenticationEntryPoint`
Wrong password / invalid or expired token	`401`	entry point (`WWW-Authenticate` header)
Logged in, missing role	`403`	`AccessDeniedHandler`
CSRF token missing	`403`	`CsrfFilter` → denied handler
Method security failure	`403`	`AccessDeniedException` from the proxy

- `ExceptionTranslationFilter` catches `AuthenticationException` and `AccessDeniedException` thrown further down and turns them into these responses.
- An anonymous user who is denied is treated as *not authenticated*: 401, not 403.
- Custom filters: extend `OncePerRequestFilter`, register with `addFilterBefore/After/At`. Do **not** also annotate it `@Component`, or Boot registers it a second time outside the security chain.
- Common uses: request ids, rate limiting, API-key checks, tenant resolution.

⚠ The error page dispatch (`/error`) also goes through the chain. Either `permitAll()` it or use `dispatcherTypeMatchers(DispatcherType.ERROR)`, otherwise a 500 can turn into a confusing 401.

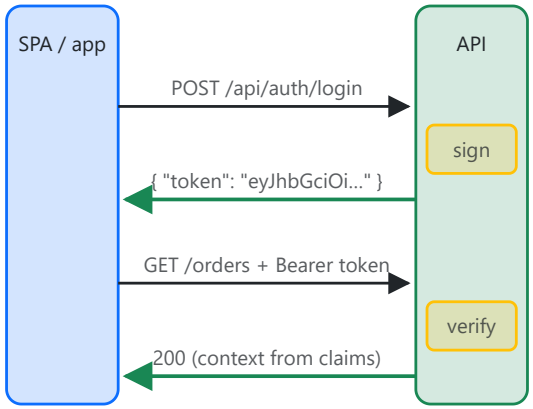
Two ways to remember who you are: session cookie or bearer token

Session-based (server state)



logout = invalidate session
scaling = sticky sessions or Spring Session

Stateless (JWT bearer token)



no server memory per user, any instance can verify
logout = client drops token (or revocation list)

	Session	Stateless JWT
State	server (HttpSession)	inside the token
Transport	cookie (automatic)	`Authorization` header
CSRF risk	yes, needs a token	no (header is not auto-sent)
XSS risk	HttpOnly cookie helps	token in JS memory is exposed
Revocation	instant (invalidate)	hard: short expiry + refresh
Scaling	sticky / shared store	trivial, verify anywhere
Best for	server-rendered web apps	REST APIs, SPAs, mobile, microservices

Hybrid is common: a SPA served from the same origin keeps the session cookie and adds a CSRF token; third-party clients and mobile apps get JWTs.

Configuring the two modes in Spring Security

```java
// A. Session-based form login for the admin UI
@Bean @Order(2)
SecurityFilterChain web(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(a -> a
            .requestMatchers("/login", "/css/**").permitAll()
            .anyRequest().authenticated())
        .formLogin(f -> f.loginPage("/login")
                         .defaultSuccessUrl("/admin"))
        .logout(l -> l.logoutUrl("/logout")
                      .deleteCookies("JSESSIONID")
                      .invalidateHttpSession(true))
        .sessionManagement(s -> s
            // new session id after login (fixation defence)
            .sessionFixation(f -> f.migrateSession())
            .maximumSessions(1)             // one device at a time
            .maxSessionsPreventsLogin(false))
        .rememberMe(r -> r.key(env.getProperty("REMEMBER_KEY"))
                          .tokenValiditySeconds(14 * 24 * 3600));
    return http.build();
}
```

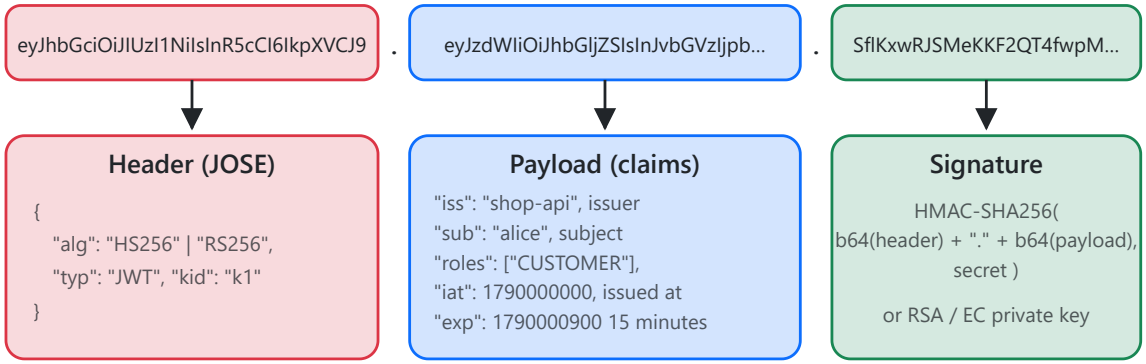
```yaml
# application.yml: cookie hardening for the session
server:
  servlet:
    session:
      timeout: 30m
      cookie:
        http-only: true       # JS cannot read it
        secure: true          # HTTPS only
        same-site: lax        # not sent on cross-site POST
```

```java
// B. Stateless JWT for the REST API
@Bean @Order(1)
SecurityFilterChain api(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")
        .authorizeHttpRequests(a -> a
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated())
        // never create or use an HttpSession
        .sessionManagement(s -> s
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        // no cookie-based login → CSRF not applicable
        .csrf(csrf -> csrf.disable())
        .httpBasic(b -> b.disable())
        .formLogin(f -> f.disable())
        // validate "Authorization: Bearer " on every request
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
    return http.build();
}
```

`SessionCreationPolicy`	**Meaning**
`ALWAYS`	create a session on every request
`IF_REQUIRED` (default)	create when something needs it (login)
`NEVER`	never create, but use an existing one
`STATELESS`	ignore sessions entirely; context lives only for the request

⚠ With `STATELESS` every request must carry its own credentials. Tests that log in once and expect to stay logged in will fail; that is the intended behaviour.

JWT anatomy: `header.payload.signature`



base64url is encoding, not encryption: anyone can read the claims. Never put secrets in a JWT.

The signature guarantees integrity and origin: change one character of the payload and verification fails.

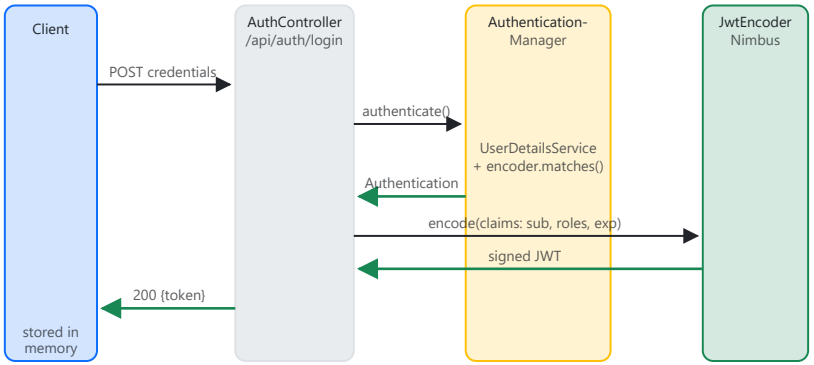
Signed = JWS (what we use). Encrypted = JWE (rare). Both follow the JWT (RFC 7519) format.

Claim	Meaning	Checked by
`iss`	who issued it	issuer validator
`sub`	the user id	becomes `getName()`
`aud`	intended API	audience validator
`exp` / `nbf`	expiry / not before	timestamp validator (60 s skew)
`iat`, `jti`	issued at, unique id	revocation lists
`roles`, `scope`	custom / standard authorities	your converter

Algorithm	Key	Use when
`HS256` (HMAC)	one shared secret, ≥ 32 bytes	issuer and verifier are the same app
`RS256` / `ES256`	private key signs, public key verifies	many services verify, only the auth server signs
`none`	must be rejected; a classic attack	

Keep access tokens short-lived (5 to 15 minutes). A stolen token is valid until `exp`; nothing on the server can take it back unless you keep a revocation list.

Issuing: the login endpoint authenticates, then signs a token



💡 The login endpoint is the **only** place that ever sees a password. Everything after it works with the token.

```java
@Configuration
public class JwtConfig {
    // HS256: one secret shared by encoder and decoder (>= 32 bytes)
    @Bean
    SecretKey jwtKey(@Value("${JWT_SECRET}") String secret) {
        return new SecretKeySpec(
            secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256");
    }
    @Bean
    JwtEncoder jwtEncoder(SecretKey key) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(key));
    }
    @Bean
    AuthenticationManager authManager(UserDetailsService uds,
                                      PasswordEncoder enc) {
        var provider = new DaoAuthenticationProvider(uds);
        provider.setPasswordEncoder(enc);
        return new ProviderManager(provider);
    }
}

@RestController @RequestMapping("/api/auth")
public class AuthController {
    private final AuthenticationManager authManager;
    private final JwtEncoder encoder;
    // constructor omitted

    public record LoginRequest(@NotBlank String username,
                               @NotBlank String password) {}
    public record TokenResponse(String token, long expiresIn) {}

    @PostMapping("/login")
    public TokenResponse login(@RequestBody @Valid LoginRequest r) {
        Authentication auth = authManager.authenticate(
            UsernamePasswordAuthenticationToken
                .unauthenticated(r.username(), r.password()));
        Instant now = Instant.now();
        var claims = JwtClaimsSet.builder()
            .issuer("shop-api").subject(auth.getName())
            .issuedAt(now).expiresAt(now.plus(15, ChronoUnit.MINUTES))
            .claim("roles", auth.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority).toList())
            .build();
        var header = JwsHeader.with(MacAlgorithm.HS256).build();
        String token = encoder.encode(
            JwtEncoderParameters.from(header, claims)).getTokenValue();
        return new TokenResponse(token, 900);
    }
}
```

Validation: the resource server verifies signature, expiry and claims

```java
@Bean
SecurityFilterChain api(HttpSecurity http,
                        JwtAuthenticationConverter conv) throws Exception {
    http.securityMatcher("/api/**")
        .authorizeHttpRequests(a -> a
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated())
        .sessionManagement(s -> s
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .csrf(c -> c.disable())
        // BearerTokenAuthenticationFilter + JwtDecoder + validators
        .oauth2ResourceServer(o -> o.jwt(j -> j
            .jwtAuthenticationConverter(conv)));
    return http.build();
}

// Verifies HS256 signature, exp/nbf (60 s skew) and the issuer
@Bean
JwtDecoder jwtDecoder(SecretKey key) {
    var decoder = NimbusJwtDecoder.withSecretKey(key)
        .macAlgorithm(MacAlgorithm.HS256).build();
    decoder.setJwtValidator(JwtValidators
        .createDefaultWithIssuer("shop-api"));
    return decoder;
}

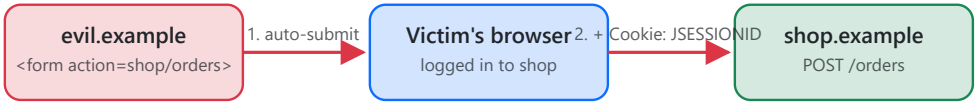
// Map the "roles" claim to GrantedAuthority (no SCOPE_ prefix)
@Bean
JwtAuthenticationConverter jwtAuthenticationConverter() {
    var roles = new JwtGrantedAuthoritiesConverter();
    roles.setAuthoritiesClaimName("roles");
    roles.setAuthorityPrefix("");        // claim already has ROLE_
    var conv = new JwtAuthenticationConverter();
    conv.setJwtGrantedAuthoritiesConverter(roles);
    return conv;
}
```

- `spring-boot-starter-oauth2-resource-server` brings Nimbus JOSE and the `BearerTokenAuthenticationFilter`: it reads `Authorization: Bearer`, decodes, validates and fills the `SecurityContext` with a `JwtAuthenticationToken`.
- **RS256** variant: sign with the private key, verify with `NimbusJwtDecoder.withPublicKey(pub)` or `withJwkSetUri(...)` when the issuer publishes its keys (Keycloak, Auth0, Entra ID).
- Expired or tampered token → `InvalidBearerTokenException` → `401` with `WWW-Authenticate: Bearer error="invalid_token"`.
- **Refresh tokens**: a second, long-lived, opaque token stored in the DB (hashed) and exchanged at `/api/auth/refresh` for a new access token. Rotate it on every use and revoke the whole family on reuse.

Approach	Dependency	Verdict
Spring resource server (Nimbus)	`spring-boot-starter-oauth2-resource-server`	**Recommended**: filter, validators, converter and test support included
jjwt 0.12.x	`io.jsonwebtoken:jjwt-api/impl/jackson`	Fine for issuing; you write the filter and the error handling yourself

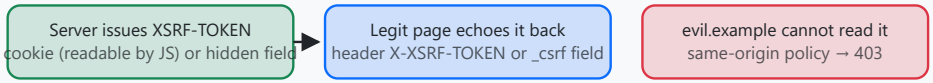
```java
// jjwt 0.12 for comparison (issue + parse)
String jwt = Jwts.builder().subject("alice")
    .claim("roles", List.of("ROLE_CUSTOMER"))
    .expiration(Date.from(now.plusSeconds(900)))
    .signWith(key).compact();
Claims c = Jwts.parser().verifyWith(key).build()
    .parseSignedClaims(jwt).getPayload();
```

CSRF: the browser sends your cookie to whoever asks



Without defence: the shop sees a valid session and places the order.

Defence: synchronizer token (CsrfFilter)



Applies to state-changing methods only (POST, PUT, PATCH, DELETE). GET must never change state.

SameSite=Lax cookies help too, but do not replace the token for browsers you do not control.

```
// 1. Server-rendered forms: the default is on. Thymeleaf/JSP
//    with the Spring tag libs add the hidden _csrf field for you.
http.csrf(Customizer.withDefaults());

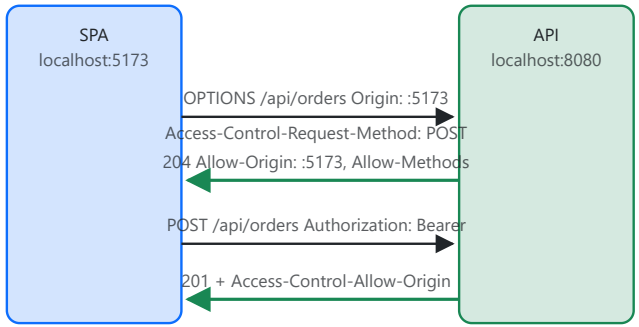
// 2. SPA on the same origin with a session cookie:
//    token in a readable cookie, JS sends it back as a header
http.csrf(csrf -> csrf
    .csrfTokenRepository(
        CookieCsrfTokenRepository.withHttpOnlyFalse())
    .csrfTokenRequestHandler(
        new CsrfTokenRequestAttributeHandler())
    .ignoringRequestMatchers("/api/auth/login"));

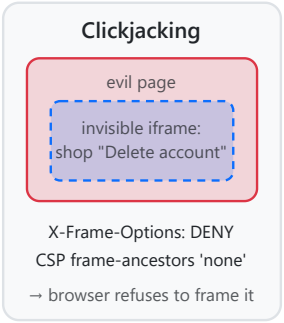
// 3. Pure stateless API (JWT in the Authorization header):
//    no cookie is sent automatically, so CSRF does not apply
http.csrf(csrf -> csrf.disable());
```

```
// SPA side (fetch): read the cookie, send it as a header
const xsrf = document.cookie.match(/XSRF-TOKEN=([^;]+)/)?.[1];
await fetch('/api/orders', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json',
             'X-XSRF-TOKEN': decodeURIComponent(xsrf) },
  credentials: 'same-origin',
  body: JSON.stringify(order)
});
```

⚠ Disabling CSRF is only correct when **no** browser-automatic credential (cookie, Basic auth cached by the browser) is accepted by that filter chain. Write the reason down in the config comment; graders and auditors will ask.

CORS, clickjacking and the security headers Spring writes for you





CORS relaxes the same-origin policy for origins you list. It is not authentication.

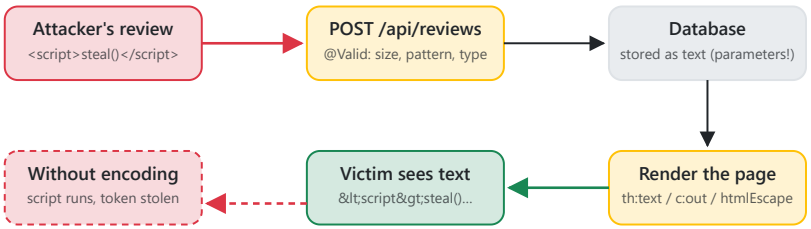
```
@Bean
CorsConfigurationSource corsConfigurationSource() {
    var cfg = new CorsConfiguration();
    cfg.setAllowedOrigins(List.of("http://localhost:5173"));
    cfg.setAllowedMethods(List.of("GET","POST","PUT","DELETE"));
    cfg.setAllowedHeaders(List.of("Authorization","Content-Type"));
    cfg.setExposedHeaders(List.of("Location"));
    cfg.setMaxAge(Duration.ofHours(1));
    var src = new UrlBasedCorsConfigurationSource();
    src.registerCorsConfiguration("/api/**", cfg);
    return src;
}
// in the chain: http.cors(Customizer.withDefaults());
```

```
http.headers(h -> h
    // clickjacking: refuse to be framed
    .frameOptions(f -> f.deny())
    // force HTTPS for a year, incl. subdomains
    .httpStrictTransportSecurity(hsts -> hsts
        .includeSubDomains(true).maxAgeInSeconds(31_536_000))
    // CSP: scripts only from us, no inline, no framing
    .contentSecurityPolicy(csp -> csp.policyDirectives(
        "default-src 'self'; script-src 'self'; "
      + "object-src 'none'; frame-ancestors 'none'; "
      + "base-uri 'self'"))
    .referrerPolicy(r -> r.policy(
        ReferrerPolicy.STRICT_ORIGIN_WHEN_CROSS_ORIGIN))
    .permissionsPolicyHeader(p -> p.policy(
        "camera=(), microphone=(), geolocation=()")));
```

Header	Default	Protects against
`X-Content-Type-Options: nosniff`	on	MIME sniffing of uploads
`X-Frame-Options: DENY`	on	clickjacking
`Cache-Control: no-store`	on	cached secrets in shared browsers
`Strict-Transport-Security`	on (HTTPS only)	downgrade to HTTP
`Content-Security-Policy`	off	XSS payloads, framing
`Referrer-Policy`, `Permissions-Policy`	off	URL leaks, device access

Verify with `curl -I https://shop.example/` or the browser's Network tab. A `*` in `Access-Control-Allow-Origin` together with credentials is refused by browsers, and listing every origin as allowed defeats the purpose.

Validate what comes in, encode what goes out



Injection is one family: SQL, HTML/JS (XSS), OS command, LDAP, log lines.

Rule 1: never build a command/query/page by concatenating untrusted text.

Rule 2: validate for shape and size at the boundary (allow-list, not deny-list).

Rule 3: encode for the output context (HTML, attribute, JS, URL, SQL parameter).

```java
// Input: shape, size and type are checked before any logic runs
public record ReviewRequest(
    @NotNull @Positive Long productId,
    @Min(1) @Max(5) int stars,
    @NotBlank @Size(max = 1000) String text) {}

@PostMapping("/api/reviews")
@ResponseStatus(HttpStatus.CREATED)
public ReviewDto add(@RequestBody @Valid ReviewRequest r,
                     Authentication auth) {
    // stored unchanged: encoding is an OUTPUT concern
    return reviews.add(auth.getName(), r);
}

// SQL: always bound parameters (JPA / JdbcClient do this)
@Query("select r from Review r where r.product.id = :pid")
List<Review> byProduct(@Param("pid") long pid);

// Output 1: JSON API - Jackson escapes; the browser must not
//   treat the response as HTML → Content-Type + nosniff header

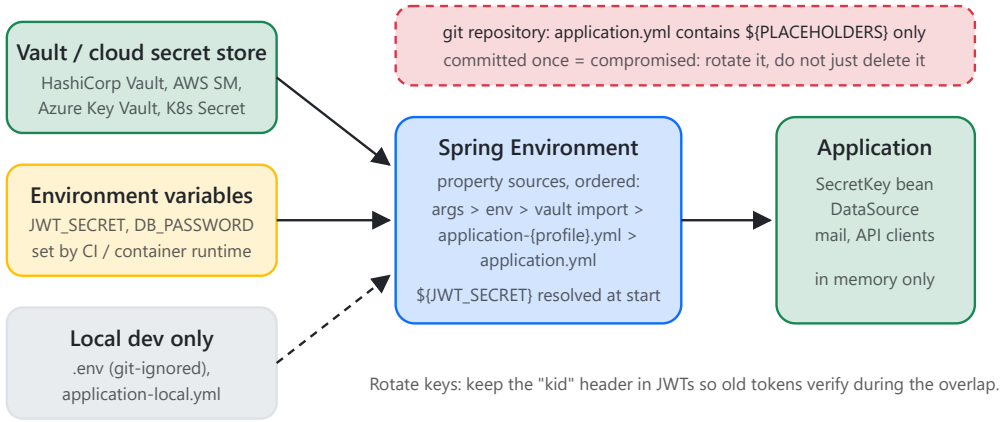
// Output 2: server-side view - the template engine escapes
//   Thymeleaf:  <p th:text="${review.text}"></p>   (th:utext = raw!)
//   JSP:        <c:out value="${review.text}"/>      (${} alone = raw!)

// Output 3: building HTML or e-mail by hand → escape explicitly
String safe = HtmlUtils.htmlEscape(review.text());
String js   = JavaScriptUtils.javaScriptEscape(review.text());

// Output 4: logs - strip line breaks so nobody forges log entries
log.info("review by {} text={}", auth.getName(),
        r.text().replaceAll("[\\r\\n]", "_"));
```

⚠ Do not "sanitize" by deleting `<script>` on input: attackers have hundreds of other payloads and you have destroyed legitimate data such as `a < b`. Validate the shape on input, encode on output, and set a CSP as the third layer.

Secrets live outside the code: environment, vault, never git



Rotate keys: keep the "kid" header in JWTs so old tokens verify during the overlap.

```yaml
# application.yml (committed) - values come from outside
spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:5432/shop
    username: ${DB_USER}
    password: ${DB_PASSWORD}
  # optional Vault import: silently skipped when Vault is absent
  config:
    import: "optional:vault://secret/shop"
shop:
  security:
    jwt-secret: ${JWT_SECRET}
    access-token-ttl: 15m
```

```java
// Typed, validated, and never logged by accident
@ConfigurationProperties("shop.security")
@Validated
public record SecurityProps(
    @NotBlank @Size(min = 32) String jwtSecret,
    @NotNull Duration accessTokenTtl) {

    @Override public String toString() {          // no leak in logs
        return "SecurityProps[ttl=" + accessTokenTtl + "]";
    }
}
```

```bash
# Run locally without committing anything
export JWT_SECRET=$(openssl rand -base64 48)
export DB_USER=shop DB_PASSWORD=dev-only
./mvnw spring-boot:run
# CI / Docker: pass with -e or a secrets manager, never in the image
```

Checklist: `.gitignore` has `.env` and `*-local.yml`; `git log -p | grep -i secret` is empty; a pre-commit secret scanner (gitleaks) runs in CI; Actuator `/env` is off or masked (`management.endpoint.env.show-values=never`).

Logging and auditing: know when someone tries the door

```java
// Spring Security publishes events; Boot registers the publisher
// (DefaultAuthenticationEventPublisher) automatically.
@Component
public class SecurityEventLogger {
    private static final Logger log =
        LoggerFactory.getLogger("SECURITY");   // dedicated logger

    @EventListener
    public void onSuccess(AuthenticationSuccessEvent e) {
        log.info("login.ok user={} type={}",
            e.getAuthentication().getName(),
            e.getAuthentication().getClass().getSimpleName());
    }

    @EventListener
    public void onFailure(AbstractAuthenticationFailureEvent e) {
        // bad password, locked, disabled, invalid JWT ... all here
        log.warn("login.failed user={} reason={} ip={}",
            e.getAuthentication().getName(),
            e.getException().getClass().getSimpleName(),
            WebAuthenticationDetails.class.isInstance(
                e.getAuthentication().getDetails())
              ? ((WebAuthenticationDetails)
                  e.getAuthentication().getDetails()).getRemoteAddress()
              : "?");
    }

    @EventListener
    public void onDenied(AuthorizationDeniedEvent<?> e) {
        log.warn("access.denied user={} object={}",
            e.getAuthentication().get().getName(), e.getObject());
    }
}
```

- **What to log**: login success and failure, lockouts, password changes, token refresh and revocation, access denied, admin actions. With *who*, *when*, *from where*, *what*.
- **What never to log**: passwords, full tokens, session ids, card numbers. Log the token's `jti` or the last 4 characters at most.
- **Structure**: key=value or JSON lines with a request id (MDC) so a security incident can be reconstructed across services.
- **Audit table** for events that must survive log rotation (who changed a role, who exported customer data): append-only, written in a `REQUIRES_NEW` transaction so a rollback of the business operation does not erase the attempt.
- **Alert** on patterns: 20 failures for one user in a minute, one IP hitting 100 usernames, a login from a new country.

```
10:14:02.117Z WARN [SECURITY] rid=7f3c…
  login.failed user=alice reason=BadCredentials
  ip=10.0.0.9
10:14:05.402Z INFO [SECURITY] rid=8a10…
  login.ok user=alice type=UsernamePassword…
10:15:31.008Z WARN [SECURITY] rid=91be…
  access.denied user=alice object=OrderService.ship(42)
```

Publish authorization events too: `@Bean AuthorizationEventPublisher` → `SpringAuthorizationEventPublisher`. Without it only failures on the filter level are visible.

Testing: prove the rules with `spring-security-test`

```
// pom.xml: spring-security-test (test scope) is pulled in by
// spring-boot-starter-security's test support; add it explicitly
// if the IDE cannot resolve SecurityMockMvcRequestPostProcessors.

@WebMvcTest(OrderController.class)
@Import(SecurityConfig.class)        // use the real filter chain
class OrderControllerSecurityTest {
    @Autowired MockMvc mvc;
    @MockitoBean OrderService orders;
    @MockitoBean JwtDecoder jwtDecoder;  // not called by jwt() below

    @Test
    void anonymousGets401() throws Exception {
        mvc.perform(get("/api/orders/1"))
           .andExpect(status().isUnauthorized())
           .andExpect(header().exists("WWW-Authenticate"));
    }

    @Test
    @WithMockUser(username = "alice", roles = "CUSTOMER")
    void customerCannotShip() throws Exception {
        mvc.perform(post("/api/orders/1/ship").with(csrf()))
           .andExpect(status().isForbidden());
        verifyNoInteractions(orders);
    }

    @Test
    void jwtWithAdminRoleCanShip() throws Exception {
        mvc.perform(post("/api/orders/1/ship")
                .with(jwt()
                    .jwt(j -> j.subject("bob")
                               .claim("roles", List.of("ROLE_ADMIN")))
                    .authorities(new SimpleGrantedAuthority("ROLE_ADMIN"))))
           .andExpect(status().isNoContent());
        verify(orders).ship(1L);
    }
}
```

```
// Method security is tested at the service level, no HTTP needed
@SpringBootTest
class OrderServiceSecurityTest {
    @Autowired OrderService service;
    @MockitoBean OrderRepository repo;

    @Test
    @WithMockUser(username = "alice", roles = "CUSTOMER")
    void ownerReadsOwnHistory() {
        service.history("alice");              // passes
    }

    @Test
    @WithMockUser(username = "alice", roles = "CUSTOMER")
    void ownerCannotReadOthers() {
        assertThrows(AccessDeniedException.class,
            () -> service.history("bob"));
    }

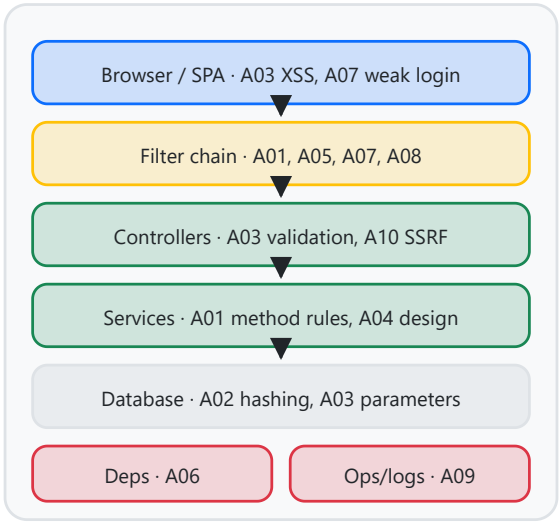
    @Test
    void noUserAtAll() {                       // no annotation
        assertThrows(AuthenticationCredentialsNotFoundException.class,
            () -> service.history("alice"));
    }
}

// End-to-end: @SpringBootTest(webEnvironment = RANDOM_PORT) with
// TestRestTemplate: POST /api/auth/login, then GET /api/orders
// with "Authorization: Bearer " + token → 200 (see Lab-05, Task 4)
```

Helper	Gives you
`@WithMockUser`, `@WithUserDetails`	a context with a fake or a loaded user
`with(jwt())`, `with(httpBasic())`	request post-processors that set up the auth for MockMvc
`with(csrf())`	a valid CSRF token when the chain requires it
`@WithAnonymousUser`	explicit anonymous, useful for 401 assertions

OWASP Top 10 (2021) mapped to the controls of this chapter

#	Risk	Shop example	Spring / Java control
A01	Broken Access Control	customer reads `/api/orders/42` of someone else	`authorizeHttpRequests`, `@PreAuthorize` owner check, `denyAll()` default
A02	Cryptographic Failures	MD5 passwords, HTTP, JWT with 8-byte secret	bcrypt/argon2, HSTS, ≥ 256-bit HMAC key or RS256, TLS everywhere
A03	Injection	SQL from search box, XSS in review	JPA/JdbcClient parameters, `@Valid`, template escaping, CSP
A04	Insecure Design	no lockout, unlimited coupons	threat modelling, rate limits, lockout, business-rule tests
A05	Security Misconfiguration	CSRF off everywhere, Actuator open, stack traces in JSON	separate filter chains, headers, `ProblemDetail` without traces, Actuator secured
A06	Vulnerable Components	old jjwt, Log4Shell-era libs	Boot BOM upgrades, `dependency-check` / Dependabot in CI
A07	Identification & Auth Failures	weak passwords, tokens that never expire	password policy, short `exp`, refresh rotation, session fixation protection
A08	Software & Data Integrity	unsigned JWT accepted, `alg: none`	Nimbus decoder pins the algorithm; signed build artefacts
A09	Logging & Monitoring Failures	nobody notices 10 000 failed logins	authentication events, audit table, alerts
A10	Server-Side Request Forgery	"fetch product image from URL" hits internal metadata service	allow-list of hosts, no redirects, network egress rules



💡 Use the list as a review checklist for every feature: for each row, where is the control and which test proves it? The OWASP ASVS and Cheat Sheet Series give the detailed requirements.

WRAP-UP

Best practices and the mistakes we see every semester

1 Deny by default

End every chain with `anyRequest().denyAll()` or `authenticated()`; open specific endpoints with `permitAll()`. Never `permitAll()` on `/**` "to make the tests pass".

2 Hash with bcrypt or argon2

`DelegatingPasswordEncoder`, cost 10 to 12, generic "bad credentials" message, lockout after repeated failures. `{noop}` only in a demo that is deleted the same day.

3 Two layers of authorization

URL rules for the coarse fence, `@PreAuthorize` on services for business rules and owner checks. Remember the proxy: public bean methods only.

4 Short JWTs, strong keys

15-minute access tokens with `exp`, `iss`, `sub`; refresh tokens stored hashed with rotation. HS256 key ≥ 32 random bytes from the environment, or RS256 when several services verify.

5 CSRF decision per chain

Cookie-based login: keep CSRF on (`CookieCsrfTokenRepository` for SPAs). Bearer-only stateless API: disable it and write down why. Never mix the two in one chain.

6 Validate in, encode out

`@Valid` DTOs with sizes and patterns; bound SQL parameters; template escaping (`th:text`, `c:out`); CSP header as the backstop. No "sanitizing" by deleting tags.

7 Secrets from the environment

`${JWT_SECRET}` placeholders in git, values from env vars or Vault, `.env` ignored, secret scanner in CI. A committed secret is rotated, not deleted.

8 Log events, not secrets

Authentication success/failure and access denied to a `SECURITY` logger with request id and IP. Never the password, never the whole token.

9 Test the negative cases

Every rule gets a 401 test, a 403 test and a 200 test: `@WithMockUser`, `jwt()`, `csrf()`. An expired token test. Security that is not tested regresses silently.

⊗ **Classic mistakes:** checking roles in the controller with `if` instead of rules · storing the JWT in `localStorage` next to an XSS hole · `hasRole("ROLE_ADMIN")` (double prefix, never matches) · returning stack traces in 500 responses · `Access-Control-Allow-Origin: *` with credentials · testing only the happy path.

Chapter quiz 10 questions

1. A logged-in customer calls DELETE /api/products/7, which requires ROLE_ADMIN. Which status code does Spring Security return?

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found

2. Why are MD5 or plain SHA-256 unsuitable for storing passwords?

- They produce hashes that are too long for a VARCHAR column
- They are fast, so billions of guesses per second can be tried offline
- They are not supported by Spring Security
- They cannot be salted

3. What does hasRole("ADMIN") actually check?

- An authority named exactly ADMIN
- An authority named ROLE_ADMIN
- A claim named admin in the JWT
- The username admin

4. Which statement about @PreAuthorize on a service method is correct?

- It works on private methods too
- It is evaluated only for HTTP requests
- It is applied through a proxy, so self-invocation bypasses it
- It replaces the need for URL rules

5. In the filter chain, which component picks the SecurityFilterChain that applies to a request?

- DispatcherServlet
- FilterChainProxy
- AuthorizationFilter
- DelegatingFilterProxy

6. Why is CSRF protection usually disabled for a stateless JWT API?

- Because JWTs are encrypted
- Because the token travels in the Authorization header, which browsers never add automatically
- Because SessionCreationPolicy.STATELESS already blocks cross-site requests
- Because CSRF only affects GET requests

7. A JWT payload contains {"sub":"alice","password":"secret"}. What is the problem?

- Nothing, the signature protects the payload
- The payload is only base64url-encoded, so anyone holding the token can read it
- JWTs may not contain more than two claims
- The sub claim must be numeric

8. What happens when a client presents an expired JWT to an endpoint protected by oauth2ResourceServer().jwt()?

- The token is silently refreshed
- 401 with WWW-Authenticate: Bearer error="invalid_token"
- 403 Forbidden
- The request proceeds as anonymous with a warning

9. A product review containing <script>steal()</script> is stored. Which measure prevents it from executing on another customer's screen?

- Deleting the word script on input
- Storing the review in a NoSQL database
- HTML-encoding the text when rendering (th:text, c:out, HtmlUtils.htmlEscape)
- Using HTTPS

10. Where should the JWT signing secret live in a Spring Boot project?

- Hard-coded in JwtConfig so it cannot be changed by mistake
- In application.yml committed to git
- In an environment variable or Vault, referenced as ${JWT_SECRET}
- In the README so the team can find it

WRAP-UP

Summary

Authentication answers "who?", authorization "what may you do?"; both meet in the per-request `SecurityContext`.

Passwords are stored as slow salted hashes (bcrypt/argon2) behind a `PasswordEncoder` and a JPA-backed `UserDetailsService`.

Roles are authorities with a `ROLE_` prefix; URL rules plus `@PreAuthorize`/`@PostAuthorize` give two layers.

Spring Security is a filter chain: `SecurityFilterChain` beans with the lambda DSL, first match wins, 401 vs 403 via entry point and denied handler.

Sessions keep state on the server (cookie, CSRF token needed); stateless APIs carry a JWT in the `Authorization` header.

A JWT is `header.payload.signature`; issue with `JwtEncoder`, validate as an OAuth2 resource server, keep `exp` short, rotate refresh tokens.

CSRF tokens for cookie-based apps, CORS allow-lists, `frame-ancestors`, HSTS and CSP headers protect the browser side.

Validate input shape with `@Valid`, bind SQL parameters, encode output for its context; secrets come from the environment or Vault.

Log authentication events, test 401/403/200 with `@WithMockUser` and `jwt()`, review features against the OWASP Top 10.

Open Lab-05: 5 tasks + 1 challenge

Next chapter: 06 · Testing in Java Web Applications (JUnit, Mockito)